

Лабораторная работа №8

Основы информационной безопасности

Панина Жанна Валерьевна

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
4	Выполнение лабораторной работы	9
5	Ответы на контрольные вопросы	12
6	Выводы	14

Список иллюстраций

4.1	Функции, использованные в работе	9
4.2	Вывод программы	10

Список таблиц

1 Цель работы

Освоить на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом

2 Задание

Два текста кодируются одним ключом (однократное гаммирование). Требуется не зная ключа и не стремясь его определить, прочитать оба текста. Необходимо разработать приложение, позволяющее шифровать и дешифровать тексты P_1 и P_2 в режиме однократного гаммирования. Приложение должно определить вид шифротекстов C_1 и C_2 обоих текстов P_1 и P_2 при известном ключе ; Необходимо определить и выразить аналитически способ, при котором злоумышленник может прочитать оба текста, не зная ключа и не стремясь его определить.

3 Теоретическое введение

Исходные данные.

Две телеграммы Центра:

P_1 = НаВашиходящийот1204

P_2 = ВСеверныйфилиалБанка

Ключ Центра длиной 20 байт: $K = 05\ 0C\ 17\ 7F\ 0E\ 4E\ 37\ D2\ 94\ 10\ 09\ 2E\ 22\ 57\ FF\ C8\ 0B\ B2\ 70\ 54$

Шифротексты обеих телеграмм можно получить по формулам режима однократного гаммирования:

$$C_1 = P_1 \oplus K,$$

$$C_2 = P_2 \oplus K. \quad (8.1)$$

Открытый текст можно найти, зная шифротекст двух телеграмм, зашифрованных одним ключом. Для это оба равенства (8.1) складываются по модулю 2. Тогда с учётом свойства операции XOR

$$1 \oplus 1 = 0, 1 \oplus 0 = 1 \quad (8.2)$$

получаем:

$$C_1 \oplus C_2 = P_1 \oplus K \oplus P_2 \oplus K = P_1 \oplus P_2.$$

Предположим, что одна из телеграмм является шаблоном — т.е. имеет текст фиксированный формат, в который вписываются значения полей. Допустим, что злоумышленнику этот формат известен. Тогда он получает достаточно много пар

$C_1 \oplus C_2$ (известен вид обеих шифровок). Тогда зная P_1 и учитывая (8.2), имеем:

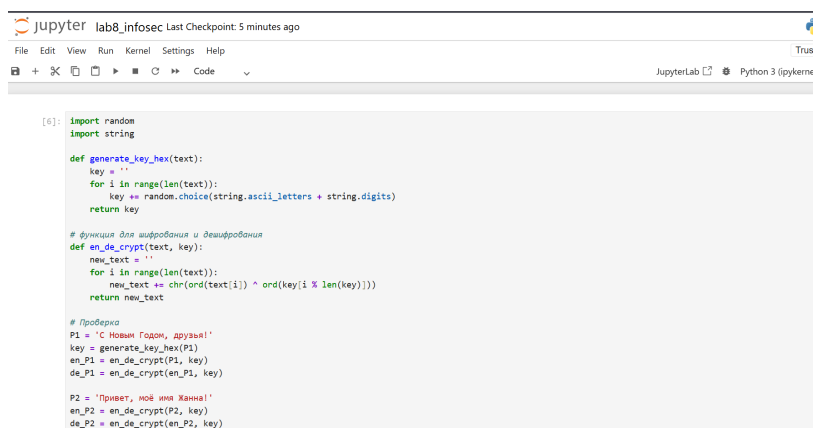
$$C_1 \oplus C_2 \oplus P_1 = P_1 \oplus P_2 \oplus P_1 = P_2. (8.3)$$

Таким образом, злоумышленник получает возможность определить те символы сообщения P_2 , которые находятся на позициях известного шаблона сообщения P_1 . В соответствии с логикой сообщения P_2 , злоумышленник имеет реальный шанс узнать ещё некоторое количество символов сообщения P_2 . Затем вновь используется (8.3) с подстановкой вместо P_1 полученных на предыдущем шаге новых символов сообщения P_2 . И так далее. Действуя подобным образом, злоумышленник даже если не прочитает оба сообщения, то значительно уменьшит пространство их поиска.

4 Выполнение лабораторной работы

Я выполняла лабораторную работу на языке программирования Python, и функции взяла из предыдущей лабораторной, поскольку они связаны между собой.

Используя функцию для генерации ключа, создаю ключ, затем шифрую два разных текста одним и тем же ключом (рис. 4.1).



```
[6]: import random
import string

def generate_key_hex(text):
    key = ''
    for i in range(len(text)):
        key += random.choice(string.ascii_letters + string.digits)
    return key

# функция для шифрования и дешифрования
def en_de_crypt(text, key):
    new_text = ''
    for i in range(len(text)):
        new_text += chr(ord(text[i]) ^ ord(key[i % len(key)]))
    return new_text

# Проверка
P1 = 'С Новым Годом, друзья!'
key = generate_key_hex(P1)
en_P1 = en_de_crypt(P1, key)
de_P1 = en_de_crypt(en_P1, key)

P2 = 'Привет, мой друг Анна!'
en_P2 = en_de_crypt(P2, key)
de_P2 = en_de_crypt(en_P2, key)
```

Рисунок 4.1: Функции, использованные в работе

Расшифровываю оба текста сначала с помощью одного ключа, затем предполагаю, что мне неизвестен ключ, но известен один из текстов и уже расшифровываю второй, зная шифротексты и первый текст (рис. 4.2).

```

print('Открытый текст: ', P1, '\nКлюч: ', key, '\nШифротекст: ', en_P1, '\nИсходный текст: ', de_P1)
print('Открытый текст: ', P2, '\nКлюч: ', key, '\nШифротекст: ', en_P2, '\nИсходный текст: ', de_P2)

r = en_de_crypt(en_P2, en_P1)
print('Расшифровать второй текст, зная первый: ', en_de_crypt(P1,r))
print('Расшифровать первый текст, зная второй: ', en_de_crypt(P2,r))

Открытый текст:  С Новым Годом, друзья!
Ключ:  0g8T05VuGdFwmKhrTk4Do
Шифротекст:  3GuXb0XUen0wēgñKazKqñN
Исходный текст:  С Новым Годом, друзья!
Открытый текст:  Привет, моё имя Жанна!
Ключ:  0g8T05VuGdFwmKhrTk4Do
Шифротекст:  24QASŸzUoēzŸsŸЧНАKİbVŸN
Исходный текст:  Привет, моё имя Жанна!
Расшифровать второй текст, зная первый:  Привет, моё имя Жанна!
Расшифровать первый текст, зная второй:  С Новым Годом, друзья!

```

Рисунок 4.2: Вывод программы

Листинг программы:

```

import random
import string

def generate_key_hex(text):
    key = ''
    for i in range(len(text)):
        key += random.choice(string.ascii_letters + string.digits)
    return key

# 00000000 0000 000000000000 0 0000000000000000

def en_de_crypt(text, key):
    new_text = ''
    for i in range(len(text)):
        new_text += chr(ord(text[i]) ^ ord(key[i % len(key)]))
    return new_text

# 00000000

P1 = '0 00000 00000, 000000!'
key = generate_key_hex(P1)
en_P1 = en_de_crypt(P1, key)

```

```

de_P1 = en_de_crypt(en_P1, key)

P2 = 'XXXXXXXX, XXX XXX XXXXX!'
en_P2 = en_de_crypt(P2, key)
de_P2 = en_de_crypt(en_P2, key)

print('XXXXXXXX XXXX: ', P1, '\nXXXX: ', key, '\nXXXXXXXXXXXX: ', en_P1, '
print('XXXXXXXX XXXX: ', P2, '\nXXXX: ', key, '\nXXXXXXXXXXXX: ', en_P2, '

r = en_de_crypt(en_P2, en_P1)
print('XXXXXXXXXXXX XXXXX XXXXX, XXX XXXXX: ', en_de_crypt(P1,r))
print('XXXXXXXXXXXX XXXXX XXXXX, XXX XXXXX: ', en_de_crypt(P2,r))

```

5 Ответы на контрольные вопросы

1. Как, зная один из текстов (P_1 или P_2), определить другой, не зная при этом ключа?
 - Для определения другого текста (P_2) можно просто взять зашифрованные тексты $C_1 \oplus C_2$, далее применить XOR к ним и к известному тексту: $C_1 \oplus C_2 \oplus P_1 = P_2$.
2. Что будет при повторном использовании ключа при шифровании текста?
 - При повторном использовании ключа мы получим дешифрованный текст.
3. Как реализуется режим шифрования однократного гаммирования одним ключом двух открытых текстов?
 - Режим шифрования однократного гаммирования одним ключом двух открытых текстов осуществляется путем XOR-ирования каждого бита первого текста с соответствующим битом ключа или второго текста.
4. Перечислите недостатки шифрования одним ключом двух открытых текстов
 - Недостатки шифрования одним ключом двух открытых текстов включают возможность раскрытия ключа или текстов при известном открытом тексте.
5. Перечислите преимущества шифрования одним ключом двух открытых текстов

- Преимущества шифрования одним ключом двух открытых текстов включают использование одного ключа для зашифрования нескольких сообщений без необходимости создания нового ключа и выделения на него памяти.

6 Выводы

В ходе выполнения работы я освоила на практике применение режима однократного гаммирования на примере кодирования различных исходных текстов одним ключом